CodeMigrate: What Is New

Novelty compared with existing systems and published work (review handout, test split, 160 cases)

1. The problem

When a Django project is upgraded (here 4.0 to 5.1), removed or deprecated APIs break the code. The release notes describe every change, but the tools we compared do not check a project against them: rule-based rewriters only know their hand-written fixers, and LLMs cover more but invent changes that never happened.

2. What is new in CodeMigrate

N1. Deterministic citation validator. Every LLM finding must cite a release-note entry. A rule-based check (not another LLM) accepts it only if the cited entry (a) was retrieved for that code line, (b) names the API used on that line, and (c) lies in the chosen version range. Otherwise the finding is rejected.

N2. Benchmark labelled by execution. Ground truth comes from running every case under Django 4.0.10, 4.1.13, 4.2.30, 5.0.14 and 5.1.15 and recording real errors and deprecation warnings, so the labels do not depend on our tool. 235 usable cases, 168 positive lines.

N3. Sourced knowledge base. 310 Django 4.0 to 5.2 changes parsed from the official release notes at a pinned commit (293 automatic, 13 corrected, 4 manual), each linked to its release-note section.

N4. Audit + fix of existing code. Related LLM work generates new code; CodeMigrate audits an existing project, points to exact lines, explains each change with a source link, and produces a patch.

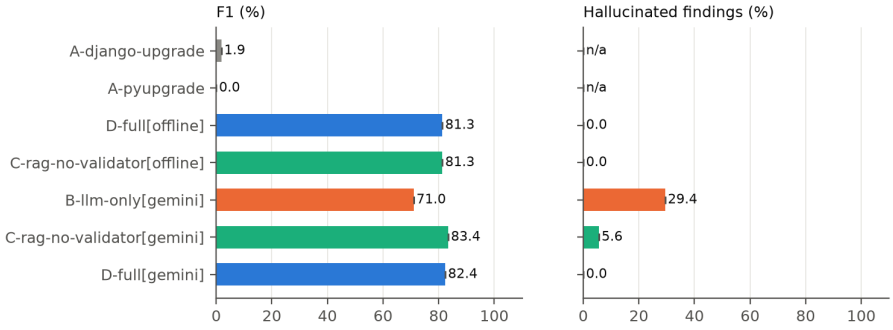
3. Comparison with existing systems

Work	Task	Docs as evidence	Checks LLM claims	Labels from execution
django-upgrade [1]	Rewrite code with fixed rules	No (hand-written fixers)	n/a (no LLM)	n/a
pyupgrade [2]	Rewrite Python syntax	No	n/a	n/a
Wang et al., ICSE'25 [3]	Deprecated API use in LLM code completion	Prompt hint (INSERTPROMPT)	No	No (API mappings)
Spracklen et al., USENIX Sec'25 [4]	Package hallucination in generated code	RAG as mitigation	Detects fake packages	No
Chen et al., 2025 [5]	RAG over API docs for code generation	Yes	No	No
CodeUpdateArena [6]	Using synthetic API updates	Update docs in prompt	No	Unit tests
GitChameleon [7]	Version-specific code generation	No	No	Yes (unit tests)
MigrateLib [8]	Migrate between different libraries	No	Static + dynamic analysis	Tests
CodeMigrate (ours)	**Find + fix breaking API uses in existing code on upgrade**	**Yes: 310 sourced release-note entries**	**Yes: deterministic citation validator**	**Yes: run under 5 Django releases**

4. Head-to-head results on the same benchmark

System	Precision	Recall	F1 [95% CI]	Hallucinated findings
A: django-upgrade (existing tool)	0.09	0.01	0.02 [0.00, 0.06]	n/a (no citations)
A: pyupgrade (existing tool)	0.00	0.00	0.00 [0.00, 0.00]	n/a (no citations)
B: Gemini only (no retrieval)	0.68	0.74	0.71 [0.63, 0.78]	29%
C: Gemini + RAG, no validator	0.77	0.92	0.83 [0.77, 0.89]	6%
D: Gemini + RAG + validator (ours)	0.78	0.88	0.82 [0.75, 0.88]	0%
D: offline reasoner + RAG + validator (ours)	0.75	0.89	0.81 [0.74, 0.88]	0%

Model: gemini-3.5-flash-lite, temperature 0. Hallucinated finding = a reported change that does not exist in the KB for that API and version range (same matcher for every system).



F1 vs. hallucination rate per system (test split).

5. What the numbers show

• Existing rewriters cover almost none of the breaking changes: django-upgrade F1 0.02, pyupgrade 0.00, vs. 0.82 for CodeMigrate.

• Gemini alone invents changes: 29% of its findings are hallucinated. Retrieval (RAG) lowers this to 6%; the validator removes the rest (0%).

• Cost of the validator: it rejected 8 Gemini claims; recall drops from 0.92 to 0.88 while precision rises from 0.77 to 0.78. The F1 difference (-0.011, 95% CI [-0.035, 0.011]) is not significant, so hallucinations go to zero at no significant accuracy cost.

• The offline reasoner (no API) reaches F1 0.81, so the tool still works without an LLM.

6. Example: a hallucination the validator caught

On a model with `Meta.unique_together`, Gemini (with RAG) reported that this option is removed. The retrieved evidence was about `index_together`, and Gemini cited no entry. The validator marked it **UNSUPPORTED_CLAIM** and dropped it. In another case Gemini cited the `TimestampSigner` removal for code using `Signer`; the symbol check rejected it as **UNVERIFIED**. All rejections are logged in `results/_runs/test-D-full[gemini]-s0.rejected.jsonl`.

7. Published results of related work (for context)

Work	Reported result (from the paper's abstract)
Spracklen et al. [4]	16 code LLMs, 576,000 samples: hallucinated packages at least 5.2% (commercial) and 21.7% (open-source) of the time.
Chen et al. [5]	RAG over API docs improves code generation by 83% to 220% on 4 less common Python libraries.
CodeUpdateArena [6]	Putting update docs in the prompt does not help open-source code LLMs use the changed API.
GitChameleon [7]	GPT-4o pass@10 of 39.9% on version-specific problems (43.7% with error feedback).
MigrateLib [8]	32% of library migrations fully correct on 717 real applications.

These numbers are on different tasks and datasets, so they are not head-to-head scores. They show the gap CodeMigrate targets: LLMs use outdated APIs and hallucinate, and adding documentation alone does not fix this. Our head-to-head comparison is Section 4.

8. Limits (stated honestly)

• One framework (Django) and one upgrade range (4.0 to 5.1).

• Execution labels only see code paths that actually ran.

• Gemini results are one run (seed 0) on the free tier.

• Most positive labels come from synthetic cases; real packages have few breaking uses.

References

[1] A. Johnson, django-upgrade. https://github.com/adamchainz/django-upgrade

[2] A. Sottile, pyupgrade. https://github.com/asottile/pyupgrade

[3] C. Wang et al., "LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-based Code Completion," ICSE 2025. arXiv:2406.09834

[4] J. Spracklen et al., "We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs," USENIX Security 2025. arXiv:2406.10279

[5] J. Chen et al., "When LLMs Meet API Documentation: Can Retrieval Augmentation Aid Code Generation Just as It Helps Developers?" arXiv:2503.15231, 2025

[6] Z. L. Liu et al., "CodeUpdateArena: Benchmarking Knowledge Editing on API Updates," arXiv:2407.06249, 2024

[7] N. Islah et al., "GitChameleon: Unmasking the Version-Switching Capabilities of Code Generation Models," arXiv:2411.05830, 2024

[8] M. Islam et al., "MigrateLib: A Tool for End-to-End Python Library Migration," arXiv:2510.08810, 2025